

3 Task specifications

Write a program in a file named `diff.py`. Your program must define the following three classes:

- `DiffCommands`
- `DiffCommandsError` (which must inherit from `Exception`)
- `OriginalNewFiles`

In the following, the first file (also called the original file) is the first argument passed to `OriginalNewFiles`, and the second file (also called the new file) is the second argument.

3.1 Class `DiffCommands`

This class represents a sequence of `diff` commands.

- It provides no public methods.
- Its constructor (`__init__`) takes as input a text file that is expected to contain a valid sequence of commands that could be produced by `diff` for some pair of files. The file must also satisfy the following formatting constraints:
 - one command per line;
 - no spaces within a line;
 - no extra or empty lines.

If the file does not satisfy these conditions, the constructor must raise a `DiffCommandsError` (1 mark) with the message:

`Cannot possibly be the commands for the diff of two files`

Seven invalid examples are provided (`wrong_1.txt` to `wrong_7.txt`).

Three valid examples are also provided:

- `diff_1.txt`: the unique sequence of commands for `file_1_1.txt` and `file_1_2.txt`
- `diff_2.txt`: one of the two possible sequences for `file_2_1.txt` and `file_2_2.txt`
- `diff_3.txt`: one of the three possible sequences for `file_3_1.txt` and `file_3_2.txt`

The class must implement the `__str__()` method (3 marks), so that calling `print()` on a `DiffCommands` object outputs its commands.

3.2 Class `OriginalNewFiles`

This class provides methods to analyse and display differences between two files. It must implement the following methods:

- `is_valid_diff()` (2 marks), which returns a Boolean;
- `print_diff()`, which prints the corresponding diff output (2 marks);
- `print_unmodified_from_original()` (1.5 marks), which prints the lines of the longest common subsequence as they appear in the original file, with `...` standing for maximal nonempty blocks of omitted lines;
- `print_unmodified_from_new()` (1.5 marks), which prints the lines of the longest common subsequence as they appear in the new file, with `...` standing for maximal nonempty blocks of omitted lines;
- `all_diff_commands()` (2 marks), which returns the list of all possible `diff` command sequences.

If the two files are identical, then `all_diff_commands()` returns a list containing a single `DiffCommands` object whose sequence of commands is empty.

3.3 Expected behaviour

An example interaction (provided below) illustrates:

- how these methods should be called;
- whether they return a value or print their result.

Methods whose names start with `print_` print their output directly and do not return a value. One additional requirement is not immediately visible in the example:

- The list returned by `OriginalNewFiles.pair_of_files.all_diff_commands()` must be sorted lexicographically according to the string representation of its elements.

```

$ python3
...
>>> from diff import *
>>> DiffCommands('wrong_1.txt')
...
diff.DiffCommandsError: Cannot possibly be the commands for the diff of two files
>>> DiffCommands('wrong_2.txt')
...
diff.DiffCommandsError: Cannot possibly be the commands for the diff of two files
>>> DiffCommands('wrong_3.txt')
...
diff.DiffCommandsError: Cannot possibly be the commands for the diff of two files
>>> DiffCommands('wrong_4.txt')
...
diff.DiffCommandsError: Cannot possibly be the commands for the diff of two files
>>> DiffCommands('wrong_5.txt')
...
diff.DiffCommandsError: Cannot possibly be the commands for the diff of two files
>>> DiffCommands('wrong_6.txt')
...
diff.DiffCommandsError: Cannot possibly be the commands for the diff of two files
>>> DiffCommands('wrong_7.txt')
...
diff.DiffCommandsError: Cannot possibly be the commands for the diff of two files
>>> DiffCommands('diff_1.txt')
<diff.DiffCommands object at ...>

```

```

>>> diff_1 = DiffCommands('diff_1.txt')
>>> print(diff_1)
1,2d0
3a2
5c4
7a7,8
10d10
13,16c13,15
>>> diff_2 = DiffCommands('diff_2.txt')
>>> print(diff_2)
1a2
>>> diff_3 = DiffCommands('diff_3.txt')
>>> print(diff_3)
0a1,2
3,5c5,11
8d13
>>> pair_of_files = OriginalNewFiles('file_1_1.txt', 'file_1_2.txt')
>>> pair_of_files.is_valid_diff(diff_1)
True
>>> pair_of_files.is_valid_diff(diff_2)
False
>>> pair_of_files.is_valid_diff(diff_3)
False
>>> pair_of_files.print_diff(diff_1)
1,2d0
< A line to delete: 1
< A line to delete: 2
3a2
> A line to insert: 1
5c4
< A line to change: 1
---
> A changed line: 1
7a7,8
> A line to insert: 2
> A line to insert: 3
10d10
< A line to delete: 3
13,16c13,15
< A line to change: 2
< A line to change: 3
< A line to change: 4
< A line to change: 5
---
> A changed line: 2
> A changed line: 3
> A changed line: 4

```

```

>>> pair_of_files.print_unmodified_from_original(diff_1)
...
A line that stays: 1
A line that stays: 2
...
A line that stays: 3
A line that stays: 4
A line that stays: 5
A line that stays: 6
...
A line that stays: 7
A line that stays: 8
...
A line that stays: 9
>>> pair_of_files.print_unmodified_from_new(diff_1)
A line that stays: 1
...
A line that stays: 2
...
A line that stays: 3
A line that stays: 4
...
A line that stays: 5
A line that stays: 6
A line that stays: 7
A line that stays: 8
...
A line that stays: 9
>>> pair_of_files.all_diff_commands()
[<diff.DiffCommands object at ...>]
>>> diffs = pair_of_files.all_diff_commands()
>>> len(diffs)
1
>>> print(diffs[0])
1,2d0
3a2
5c4
7a7,8
10d10
13,16c13,15
>>> pair_of_files = OriginalNewFiles('file_1_2.txt', 'file_1_1.txt')
>>> diffs = pair_of_files.all_diff_commands()
>>> len(diffs)
1
>>> print(diffs[0])
0a1,2
2d3
4c5
7,8d7
10a10
13,15c13,16

```

```

>>> pair_of_files = OriginalNewFiles('file_1_1.txt', 'file_1_1.txt')
>>> diffs = pair_of_files.all_diff_commands()
>>> len(diffs)
1
>>> print(diffs[0])

>>> pair_of_files = OriginalNewFiles('file_2_1.txt', 'file_2_2.txt')
>>> pair_of_files.is_valid_diff(diff_1)
False
>>> pair_of_files.is_valid_diff(diff_2)
True
>>> pair_of_files.is_valid_diff(diff_3)
False
>>> pair_of_files.print_diff(diff_2)
1a2
> A line
>>> pair_of_files.print_unmodified_from_original(diff_2)
A line
>>> pair_of_files.print_unmodified_from_new(diff_2)
A line
...
>>> diffs = pair_of_files.all_diff_commands()
>>> len(diffs)
2
>>> print(diffs[0])
0a1
>>> print(diffs[1])
1a2
>>> pair_of_files = OriginalNewFiles('file_2_2.txt', 'file_2_1.txt')
>>> diffs = pair_of_files.all_diff_commands()
>>> len(diffs)
2
>>> print(diffs[0])
1d0
>>> print(diffs[1])
2d1
>>> pair_of_files = OriginalNewFiles('file_3_1.txt', 'file_3_2.txt')
>>> pair_of_files.is_valid_diff(diff_1)
False
>>> pair_of_files.is_valid_diff(diff_2)
False
>>> pair_of_files.is_valid_diff(diff_3)
True

```

```

>>> pair_of_files.print_diff(diff_3)
0a1,2
> A line to come
> A line to come
3,5c5,11
< A line to go
< A line to go
< A line to go
---
> A line to come
> A line to come
> Line 1
> Line 2
> A line to come
> A line to come
> A line to come
8d13
< A line to go
>>> pair_of_files.print_unmodified_from_original(diff_3)
Line 1
Line 2
...
Line 3
Line 4
...
>>> pair_of_files.print_unmodified_from_new(diff_3)
...
Line 1
Line 2
...
Line 3
Line 4
>>> diffs = pair_of_files.all_diff_commands()
>>> len(diffs)
3
>>> print(diffs[0])
0a1,2
1a4,7
3,5c9,11
8d13
>>> print(diffs[1])
0a1,2
3,5c5,11
8d13
>>> print(diffs[2])
0a1,6
3,5c9,11
8d13
>>> pair_of_files = OriginalNewFiles('file_3_2.txt', 'file_3_1.txt')
>>> diffs = pair_of_files.all_diff_commands()
>>> len(diffs)
3

```

```
>>> print(diffs[0])
1,2d0
4,7d1
9,11c3,5
13a8
>>> print(diffs[1])
1,2d0
5,11c3,5
13a8
>>> print(diffs[2])
1,6d0
9,11c3,5
13a8
```